

Assignment 14

Apache NiFi, Kafka and PySpark JSON Flattening

1. Corrected Assignment Questions

- Explain the NiFi flow connecting **InvokeHTTP** to **PutFile**.
- Explain the NiFi flow connecting **InvokeHTTP** to **PutKafka**.
- How can NiFi retrieve JSON from an API and send it to a Kafka topic?
- How can PySpark read JSON from Kafka and flatten it into a tabular DataFrame?
- How can the flattened streaming data be written to CSV?

2. Understanding the NiFi Diagram

API endpoint: <https://randomuser.me/api/?results=1>

The response contains a **results** array with user information and an **info** object containing response metadata.

Scenario 1: InvokeHTTP → PutFile

Flow: InvokeHTTP → PutFile

- **InvokeHTTP** sends an HTTP request to the Random User API.
- The API returns a JSON response.
- NiFi carries the response in the content of a FlowFile.
- **PutFile** writes that content to a directory on the local file system.

Result: The API response is stored as a file. This flow does not send data to Kafka.

In the screenshot, InvokeHTTP shows about 1.39 KB of output and PutFile shows about 1.39 KB read/written, consistent with a response being passed to the file-writing processor.

Scenario 2: InvokeHTTP → PutKafka

Flow: InvokeHTTP → PutKafka

- **InvokeHTTP** calls the API.
- The API returns JSON.
- NiFi passes the response to **PutKafka**.
- **PutKafka** publishes the JSON message to the configured topic, `random_api_topic`.
- A PySpark Structured Streaming application subscribes to that topic and processes messages.

Result: The API response is sent to Kafka for downstream consumption.

In the screenshot, the lower connection has about 1.41 KB queued while PutKafka shows 0 bytes processed. Data appears to be waiting in the connection at that moment. Check that PutKafka is running, broker/topic settings are correct, and no processor error is present.

Comparison

Aspect	PutFile	PutKafka
Destination	Local file system	Kafka topic
Purpose	Store API response as a file	Publish response as a message
Next step	Read the saved file	Consume messages from Kafka

3. Understanding the API JSON

The response has two main parts:

- **results:** an array containing user records.
- **info:** metadata such as seed, result count, page and version.

Each user record contains nested objects such as **name**, **location**, **login**, **dob**, **registered**, **picture** and **id**. For example, `name.first` is nested inside `name`, while `location.street.name` is nested inside `location.street`.

Kafka initially exposes the message value as binary. Casting it to a string makes the JSON readable, but it does not yet create structured columns.

4. Complete Corrected PySpark Code

The class schema differs from the supplied API response: user fields are directly inside each results element; street is a struct; username is under login; and metadata is under info. The schema below matches the provided example.

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import *
from pyspark.sql.types import *

# 1. Create SparkSession
spark = (
    SparkSession.builder
        .appName("RandomUserKafkaStreaming")
        .config(
            "spark.jars.packages",
            "org.apache.spark:spark-sql-kafka-0-10_2.12:3.5.6"
        )
        .getOrCreate()
)

# 2. Read streaming data from Kafka
df = (
    spark.readStream
        .format("kafka")
        .option("kafka.bootstrap.servers", "localhost:9092")
        .option("subscribe", "random_api_topic")
        .option("startingOffsets", "earliest")
        .option("maxOffsetsPerTrigger", 1000)
        .load()
)

# 3. Convert Kafka key and value from binary to string
kafka_df = (
    df.withColumn("key", col("key").cast("string"))
        .withColumn("value", col("value").cast("string"))
)

# 4. Define a schema matching the API JSON
value_schema = StructType([
    StructField(
        "results",
        ArrayType(
            StructType([
                StructField("gender", StringType()),
                StructField("name", StructType([
                    StructField("title", StringType()),
                    StructField("first", StringType()),
                    StructField("last", StringType())
                ])),
                StructField("location", StructType([
                    StructField("street", StructType([
                        StructField("number", IntegerType()),
                        StructField("name", StringType())
                    ])),
                    StructField("city", StringType()),
                    StructField("state", StringType()),
                    StructField("country", StringType()),
                    StructField("coordinates", StructType([
                        StructField("latitude", StringType()),
                        StructField("longitude", StringType())
                    ]))
                ])),
                StructField("email", StringType()),
                StructField("login", StructType([
                    StructField("uuid", StringType()),
                    StructField("username", StringType())
                ])),
                StructField("dob", StructType([
                    StructField("date", StringType()),
                    StructField("age", IntegerType())
                ])),
                StructField("phone", StringType()),
                StructField("cell", StringType()),
                StructField("nat", StringType())
            ])
        ),
    StructField("info", StructType([
        StructField("seed", StringType()),
        StructField("results", IntegerType()),
    ]))
])
```

```

        StructField("page", IntegerType()),
        StructField("version", StringType())
    ]))
})

# 5. Parse JSON string into a structured column
kafka_df1 = kafka_df.withColumn(
    "value_processed",
    from_json(col("value"), value_schema)
)

# 6. Extract top-level fields
df1 = kafka_df1.select(
    col("key"),
    col("value_processed.results").alias("results"),
    col("value_processed.info.seed").alias("seed"),
    col("value_processed.info.results").alias("total_results"),
    col("value_processed.info.page").alias("page"),
    col("value_processed.info.version").alias("version")
)

# 7. Explode the results array
df2 = df1.withColumn("user", explode(col("results")))

# 8. Flatten nested user fields
df3 = df2.select(
    col("key"),
    col("seed"),
    col("total_results"),
    col("page"),
    col("version"),
    col("user.gender").alias("gender"),
    col("user.name.title").alias("name_title"),
    col("user.name.first").alias("first_name"),
    col("user.name.last").alias("last_name"),
    col("user.location.street.number").alias("street_number"),
    col("user.location.street.name").alias("street_name"),
    col("user.location.city").alias("city"),
    col("user.location.state").alias("state"),
    col("user.location.country").alias("country"),
    col("user.location.coordinates.latitude").alias("latitude"),
    col("user.location.coordinates.longitude").alias("longitude"),
    col("user.email").alias("email"),
    col("user.login.uuid").alias("user_uuid"),
    col("user.login.username").alias("username"),
    col("user.dob.date").alias("date_of_birth"),
    col("user.dob.age").alias("age"),
    col("user.phone").alias("phone"),
    col("user.cell").alias("cell"),
    col("user.nat").alias("nationality")
)

# 9. Check the flattened schema
df3.printSchema()

# 10. Write streaming DataFrame to CSV
query = (
    df3.writeStream
        .format("csv")
        .option("path", "file:///F:/data/kafka_random_api_topic_data1")
        .option(
            "checkpointLocation",
            "file:///C:/tmp/kafka_random_api_topic_checkpoint1"
        )
        .outputMode("append")
        .start()
)

query.awaitTermination()

```

Privacy note: The code intentionally excludes the API's password, password hashes and other login secrets. Include sensitive fields only when genuinely required and appropriately protected.

5. Explanation of the Flattening Steps

Step 1: Cast Kafka value to string

```
kafka_df = df.withColumn("value", col("value").cast("string"))
```

Kafka's key and value are binary columns. Casting value to string converts the message bytes into readable JSON text.

Step 2: Parse JSON with from_json()

```
from_json(col("value"), value_schema)
```

This parses the JSON string into a structured Spark column. The schema tells Spark which fields are strings, integers, arrays or nested structs.

Step 3: Extract top-level fields

```
col("value_processed.results")
col("value_processed.info.seed")
```

These expressions access the results array and fields inside the info struct.

Step 4: Convert array elements into rows

```
explode(col("results"))
```

The API returns results as an array. `explode()` creates a separate row for each array element. With `results=1`, each API response normally creates one user row.

Step 5: Select nested fields

```
col("user.name.first").alias("first_name")
col("user.location.city").alias("city")
```

Dot notation accesses nested fields. `alias()` assigns simple column names suitable for querying and tabular output.

6. Expected Flattened Output

Values vary because the API generates different users. Based on the supplied example:

gender	name_title	first_name	last_name	city	country
female	Miss	Barbara	Steward	Dublin	Ireland

Other columns include `street_number`, `street_name`, `latitude`, `longitude`, `username`, `age`, `phone`, `cell` and `nationality`. The nested JSON is now represented as ordinary columns.

7. Important Streaming Concepts

- **startingOffsets = earliest:** For a new query without an existing checkpoint, start at the earliest available Kafka offsets. With a valid checkpoint, Spark resumes from its recorded progress.
- **maxOffsetsPerTrigger = 1000:** Limits Kafka offsets processed in each micro-batch.
- **checkpointLocation:** Stores streaming progress and state needed for recovery. Keep it stable when restarting the same query.
- **outputMode(append):** Writes newly appended output rows.
- **CSV output path:** Spark writes a directory containing part files and supporting metadata, rather than one single CSV file.

8. End-to-End Data Flow

Step	Component	Purpose
1	Random User API	Returns JSON
2	NiFi — InvokeHTTP	Fetches the API response
3	NiFi — PutKafka	Publishes JSON to <code>random_api_topic</code>
4	PySpark Structured Streaming	Reads Kafka messages and parses JSON
5	Flattened DataFrame	<code>from_json</code> → <code>explode</code> → <code>select</code> nested fields
6	CSV output	Stores flat, tabular data

Key takeaway: NiFi moves the API response into Kafka. PySpark parses the JSON string, expands the results array and nested structs, and writes the resulting flat records to CSV.